

HARIHARAN G

192424156

DSAS0503

QUERY PROCESSING

ACTIVITY ON SQLITE

1. Create the Department Table (Parent Table) with attributes Department_ID
INTEGER PRIMARY KEY, Department_Name TEXT

Aim

To create a **Department** table in an SQLite database using Python, insert sample department records, and display the stored data.

Theory

A **Department** table is a **parent table** that stores information about different departments in a college. Each department is identified by a unique **Department_ID**, which acts as the **Primary Key**. This table is referenced by other tables, such as the **Student** table, through a **Foreign Key** relationship.

SQLite is a lightweight, serverless relational database management system that can be accessed using Python's built-in **sqlite3** module. It allows users to create tables, insert records, retrieve data, and manage databases efficiently.

The Department table stores the details of academic departments and serves as the foundation for maintaining relationships with other tables in the database.

Table Structure

Attribute	Data Type	Description
Department_ID	INTEGER	Unique Department ID (Primary Key)
Department_Name	TEXT	Name of the Department

Python Code

```
import sqlite3

# Connect to SQLite Database
conn = sqlite3.connect("college.db")

# Create Cursor
cursor = conn.cursor()
```

```
# Create Department Table
```

```
cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS Department(
```

```
    Department_ID INTEGER PRIMARY KEY,
```

```
    Department_Name TEXT
```

```
)
```

```
""")
```

```
# Insert Sample Records
```

```
departments = [
```

```
    (101, "CSE"),
```

```
    (102, "ECE"),
```

```
    (103, "EEE"),
```

```
    (104, "MECH"),
```

```
    (105, "CIVIL")
```

```
]
```

```
cursor.executemany(
```

```
    "INSERT OR IGNORE INTO Department VALUES (?, ?)",
```

```
    departments
```

```
)
```

```
# Save Changes
```

```
conn.commit()
```

```
# Display Records
```

```
cursor.execute("SELECT * FROM Department")
```

```
rows = cursor.fetchall()
```

```
print("Department Table")
print("-----")
print("Department_ID\tDepartment_Name")

for row in rows:
    print(row[0], "\t\t", row[1])

# Close Connection
conn.close()
```

Working of the Program

The program establishes a connection with the SQLite database named **college.db**. If the database file does not exist, SQLite automatically creates it.

A **Department** table is then created using the CREATE TABLE IF NOT EXISTS statement. This ensures that the table is created only once and prevents errors if it already exists.

Next, five sample department records (**CSE, ECE, EEE, MECH, and CIVIL**) are inserted into the Department table using the executemany() method. The INSERT OR IGNORE statement avoids inserting duplicate records if the program is executed multiple times.

After inserting the records, the changes are permanently saved using the commit() method. The program then retrieves all records from the Department table using the SELECT statement and displays them in a tabular format. Finally, the database connection is closed.

Expected Output

```
=== RESTART: C:/Users/seivi/OneDrive/Desktop
Department Table
-----
Department_ID    Department_Name
101              CSE
102              ECE
103              EEE
104              MECH
105              CIVIL
>
```

Result

The **Department** table was successfully created in the SQLite database. Sample department records were inserted, stored, and displayed successfully. The table acts as the **parent table** and can be referenced by the **Student** table through the **Department_ID** foreign key to maintain referential integrity in the database.

2. Create the Student Table (Child Table) with the following attributes:

Student_ID INTEGER PRIMARY KEY,

Name TEXT,

Age INTEGER,

Department_ID INTEGER,

CGPA REAL,

FOREIGN KEY (Department_ID) REFERENCES Department(Department_ID)

Aim

To create a **Student** table in SQLite using Python with a **foreign key** relationship to the **Department** table.

Theory

A **child table** is a table that depends on another table (called the **parent table**) through a **foreign key**.

In this database:

- **Department** is the **parent table**.
- **Student** is the **child table**.

The **Department_ID** in the Student table refers to the **Department_ID** in the Department table. This ensures that every student belongs to a valid department.

Attributes of Student Table

Attribute	Data Type	Description
Student_ID	INTEGER	Unique ID of each student (Primary Key)
Name	TEXT	Name of the student
Age	INTEGER	Age of the student
Department_ID	INTEGER	Department assigned to the student (Foreign Key)
CGPA	REAL	Student's CGPA

Key Constraints

- **Primary Key**
 - Student_ID

- Ensures each student has a unique ID.
- **Foreign Key**
 - Department_ID
 - References the Department_ID in the Department table.
 - Maintains referential integrity between both tables.

SQL Query

```
CREATE TABLE Student(  
    Student_ID INTEGER PRIMARY KEY,  
    Name TEXT,  
    Age INTEGER,  
    Department_ID INTEGER,  
    CGPA REAL,  
    FOREIGN KEY (Department_ID)  
    REFERENCES Department(Department_ID)  
);
```

Python Code

```
import sqlite3  
  
# Connect to database  
conn = sqlite3.connect("college.db")  
  
# Create cursor  
cursor = conn.cursor()  
  
# Enable Foreign Key support  
cursor.execute("PRAGMA foreign_keys = ON")  
  
# Create Department Table
```

```
cursor.execute("""  
CREATE TABLE IF NOT EXISTS Department(  
    Department_ID INTEGER PRIMARY KEY,  
    Department_Name TEXT  
)  
""")
```

Create Student Table

```
cursor.execute("""  
CREATE TABLE IF NOT EXISTS Student(  
    Student_ID INTEGER PRIMARY KEY,  
    Name TEXT,  
    Age INTEGER,  
    Department_ID INTEGER,  
    CGPA REAL,  
    FOREIGN KEY (Department_ID)  
    REFERENCES Department(Department_ID)  
)  
""")
```

```
print("Student table created successfully.")
```

Save changes

```
conn.commit()
```

Close connection

```
conn.close()
```

Output

```
>>>|  
|=== RESTART: C:/Users/selvi/OneDrive/Desktop/query processing  
|Student table created successfully.  
>>>|
```

Result

The **Student** table was successfully created as a **child table** with:

- Student_ID as the **Primary Key**
- Department_ID as the **Foreign Key** referencing the **Department** table
- Proper enforcement of the relationship between students and departments using SQLite's foreign key constraint.

This format is suitable for a **lab record or practical report**.

3.Display students from CSE department.

Aim

To retrieve and display the details of students belonging to the **Computer Science and Engineering (CSE)** department using Python and SQLite.

Theory

In a relational database, data is often stored in multiple related tables to avoid redundancy. The **Department** table stores department information, while the **Student** table stores student details along with the corresponding **Department_ID**.

To display students from a specific department, an **INNER JOIN** operation is used to combine the **Student** and **Department** tables based on the common attribute **Department_ID**. The **WHERE** clause is then used to filter and display only those students who belong to the **CSE** department.

If the **Student** table does not already contain records, sample student data is inserted before retrieving the required information. This ensures that the query produces the desired output.

Python Code

```
import sqlite3

# Connect to SQLite Database
conn = sqlite3.connect("college.db")

# Create Cursor
cursor = conn.cursor()

# Enable Foreign Key
cursor.execute("PRAGMA foreign_keys = ON")

# Insert Sample Student Records
students = [
```

```

(1, "Rahul", 20, 101, 8.9),
(2, "Anitha", 21, 102, 8.5),
(3, "Priya", 20, 101, 9.2),
(4, "Karthik", 22, 103, 8.1),
(5, "Meena", 21, 104, 8.7)
]

cursor.executemany(
    "INSERT OR IGNORE INTO Student VALUES (?, ?, ?, ?, ?)",
    students
)

conn.commit()

# Display Students from CSE Department
cursor.execute("""
SELECT Student.Student_ID,
       Student.Name,
       Student.Age,
       Department.Department_Name,
       Student.CGPA
FROM Student
INNER JOIN Department
ON Student.Department_ID = Department.Department_ID
WHERE Department.Department_Name = 'CSE'
""")

rows = cursor.fetchall()

```

```

print("Students from CSE Department")
print("-----")
print("Student_ID\tName\tAge\tDepartment\tCGPA")

for row in rows:

    print(f'{row[0]}\t\t{row[1]}\t\t{row[2]}\t\t{row[3]}\t\t{row[4]}')

# Close Connection
conn.close()

```

Working of the Program

The program establishes a connection with the **college.db** SQLite database and enables foreign key support to maintain referential integrity. It then inserts sample student records into the **Student** table using the INSERT OR IGNORE statement, which prevents duplicate entries when the program is executed multiple times.

After saving the records, the program performs an **INNER JOIN** between the **Student** and **Department** tables using the common Department_ID field. The **WHERE** clause filters the results to display only students whose department name is **CSE**. Finally, the selected student details, including Student ID, Name, Age, Department Name, and CGPA, are displayed in a tabular format, and the database connection is closed.

Expected Output

```

>> | === RESTART: C:/Users/selvi/OneDrive/Desktop/query processing/activi
>> | Students from CSE Department
>> | -----
>> | Student_ID      Name      Age      Department      CGPA
>> | 1                Rahul     20       CSE              8.9
>> | 3                Priya     20       CSE              9.2
>> |

```

Note: The output may vary depending on the student records stored in the database.

Result

The program successfully inserted sample student records into the **Student** table and displayed the details of students belonging to the **CSE department**. The use of **INNER JOIN** and the **WHERE** clause demonstrates how related tables can be combined and filtered to retrieve specific information from a relational database.

4.Count students in each department.

Aim

To count and display the total number of students in each department using Python and SQLite.

Theory

In a relational database, aggregate functions are used to perform calculations on groups of records. The **COUNT()** function is an aggregate function that returns the total number of records in a group.

The **Student** table stores student details, while the **Department** table stores department information. Since both tables are related through the `Department_ID` attribute, an **INNER JOIN** is used to combine them.

The **GROUP BY** clause groups all students based on their department name, and the **COUNT()** function calculates the total number of students in each department. This provides a summary of student distribution across all departments.

Python Code

```
import sqlite3

# Connect to SQLite Database
conn = sqlite3.connect("college.db")

# Create Cursor
cursor = conn.cursor()

# Count students in each department
cursor.execute("""
SELECT Department.Department_Name,
       COUNT(Student.Student_ID) AS Total_Students
FROM Department
```

```
LEFT JOIN Student
ON Department.Department_ID = Student.Department_ID
GROUP BY Department.Department_Name
""")
```

```
rows = cursor.fetchall()
```

```
print("Student Count in Each Department")
print("-----")
print("Department\tTotal Students")
```

```
for row in rows:
```

```
    print(f'{row[0]}\t\t{row[1]}')
```

```
# Close Connection
```

```
conn.close()
```

Working of the Program

The program connects to the **college.db** SQLite database and creates a cursor object to execute SQL commands. It performs a **LEFT JOIN** between the **Department** and **Student** tables using the common **Department_ID**. This ensures that all departments are displayed, even if a department has no students.

The **GROUP BY** clause groups the records according to the department name, and the **COUNT()** function calculates the total number of students in each department. The results are then displayed in a tabular format showing each department along with its corresponding student count. Finally, the database connection is closed.

Expected Output

```
>|  
=== RESTART: C:\Users\selvi\OneDrive\Desktop\query process  
Student Count in Each Department  
-----  
Department      Total Students  
CIVIL            0  
CSE              2  
ECE              1  
EEE              1  
MECH             1  
>|
```

Result

The program successfully counted and displayed the number of students in each department using the **COUNT()** aggregate function and the **GROUP BY** clause. It demonstrates how aggregate functions and table joins can be used to summarize and analyze data stored in a relational database.

5. Update the foreign key. For eg., change the department of a student

Aim

To update the **Department_ID (Foreign Key)** of a student in the **Student** table using Python and SQLite.

Theory

In a relational database, a **Foreign Key** establishes a relationship between two tables. The **Student** table contains the **Department_ID** as a foreign key, which references the **Department_ID** in the **Department** table.

When a student changes from one department to another, the corresponding **Department_ID** in the **Student** table must be updated. The **UPDATE** statement is used to modify the foreign key value while maintaining referential integrity. The new **Department_ID** must already exist in the **Department** table.

Python Code

```
import sqlite3

# Connect to SQLite Database
conn = sqlite3.connect("college.db")

# Create Cursor
cursor = conn.cursor()

# Change Rahul's department from CSE (101) to ECE (102)
cursor.execute("""
UPDATE Student
SET Department_ID = 102
WHERE Student_ID = 1
""")
```

```

conn.commit()

# Display Updated Student Record
cursor.execute("""
SELECT Student.Student_ID,
       Student.Name,
       Department.Department_Name,
       Student.CGPA
FROM Student
INNER JOIN Department
ON Student.Department_ID = Department.Department_ID
WHERE Student.Student_ID = 1
""")

row = cursor.fetchone()

print("Updated Student Record")
print("-----")
print("Student ID :", row[0])
print("Name      :", row[1])
print("Department :", row[2])
print("CGPA       :", row[3])

# Close Connection
conn.close()

```

Working of the Program

The program connects to the **college.db** database and creates a cursor object to execute SQL commands. It uses the **UPDATE** statement to change the Department_ID of the student with

Student_ID = 1 from **101 (CSE)** to **102 (ECE)**. After updating the record, the changes are saved using the `commit()` method.

The program then retrieves the updated student details by performing an **INNER JOIN** between the **Student** and **Department** tables. The updated information is displayed, showing the student's new department. Finally, the database connection is closed.

Expected Output

```
==== RESTART: C:/Users/selvi/OneDrive/Desktop/
Updated Student Record
-----
Student ID : 1
Name       : Rahul
Department : ECE
CGPA       : 8.9
>
=== RESTART: C:/Users/selvi/OneDrive/Desktop/
```

Result

The **Department_ID** (Foreign Key) of the selected student was successfully updated from **CSE** to **ECE**. The updated record confirms that the foreign key value has been changed while maintaining the relationship between the **Student** and **Department** tables.

6. Delete a department

Aim

To delete a department from the **Department** table using Python and SQLite.

Theory

In a relational database, the **DELETE** statement is used to remove one or more records from a table. The **Department** table stores the details of different departments and acts as the **parent table** for the **Student** table.

Before deleting a department, it is important to ensure that no student is associated with that department. If foreign key constraints are enabled, attempting to delete a department that is referenced by students will result in an error. Therefore, only a department with no dependent student records should be deleted, or the related student records must be updated or removed first.

Python Code

```
import sqlite3

# Connect to SQLite Database
conn = sqlite3.connect("college.db")

# Create Cursor
cursor = conn.cursor()

# Enable Foreign Key
cursor.execute("PRAGMA foreign_keys = ON")

# Delete CIVIL Department (Department_ID = 105)
cursor.execute("""
DELETE FROM Department
WHERE Department_ID = 105
```

```
""")

conn.commit()

# Display Remaining Departments
cursor.execute("SELECT * FROM Department")
rows = cursor.fetchall()

print("Department Table After Deletion")
print("-----")
print("Department_ID\tDepartment_Name")

for row in rows:
    print(row[0], "\t\t", row[1])

# Close Connection
conn.close()
```

Working of the Program

The program establishes a connection with the **college.db** database and enables foreign key support. It then executes the **DELETE** statement to remove the department with **Department_ID = 105 (CIVIL)** from the **Department** table. After the deletion, the changes are permanently saved using the **commit()** method. The program retrieves and displays the remaining department records to verify that the selected department has been successfully removed. Finally, the database connection is closed.

Expected Output

```
>
=== RESTART: C:/Users/selvi/OneDrive/Desktop/que
Department Table After Deletion
-----
Department_ID  Department_Name|
101            CSE
102            ECE
103            EEE
104            MECH
>
=== RESTART: C:/Users/selvi/OneDrive/Desktop/que
```

Note: If any student is assigned to **Department_ID = 105**, SQLite will display a **FOREIGN KEY constraint failed** error. In that case, first update or delete the corresponding student records before deleting the department.

Result

The selected department was successfully deleted from the **Department** table using the **DELETE** statement. The remaining department records were displayed, confirming that the deletion was completed successfully while maintaining database integrity.

7. Display the student's name, student's department and CGPA if the student's CGPA is greater than 8.

Aim

To display the **student's name, department, and CGPA** for students whose **CGPA is greater than 8** using Python and SQLite.

Theory

In a relational database, the **WHERE** clause is used to filter records based on a specified condition. The **Student** table stores the student's details, including CGPA, while the **Department** table stores the department information.

Since the department name is stored in the **Department** table, an **INNER JOIN** is used to combine the **Student** and **Department** tables using the common Department_ID. The **WHERE** clause filters the records and displays only those students whose **CGPA is greater than 8**.

Python Code

```
import sqlite3

# Connect to SQLite Database
conn = sqlite3.connect("college.db")

# Create Cursor
cursor = conn.cursor()

# Display students with CGPA greater than 8
cursor.execute("""
SELECT Student.Name,
       Department.Department_Name,
       Student.CGPA
FROM Student
```

```
INNER JOIN Department
ON Student.Department_ID = Department.Department_ID
WHERE Student.CGPA > 8
""")
```

```
rows = cursor.fetchall()
```

```
print("Students with CGPA Greater Than 8")
print("-----")
print("Name\t\tDepartment\tCGPA")
```

```
for row in rows:
```

```
    print(f'{row[0]}\t\t{row[1]}\t\t{row[2]}")
```

```
# Close Connection
```

```
conn.close()
```

Working of the Program

The program connects to the **college.db** SQLite database and creates a cursor object to execute SQL commands. It performs an **INNER JOIN** between the **Student** and **Department** tables using the common **Department_ID**. The **WHERE** clause filters the records to select only those students whose **CGPA is greater than 8**. The program then displays the student's **name**, **department**, and **CGPA** in a tabular format. Finally, the database connection is closed.

Expected Output

```
=== RESTART: C:/Users/selvi/OneDrive/Desktop/query process:
Students with CGPA Greater Than 8
-----
Name           Department    CGPA
Rahul          ECE             8.9
Anitha         ECE             8.5
Priya          CSE             9.2
Karthik        EEE             8.1
Meena          MECH            8.7
>>>
```

Note: The output may vary depending on the student records stored in the database. If you updated Rahul's department in **Question 5**, his department will appear as **ECE** instead of **CSE**.

Result

The program successfully displayed the **student's name, department, and CGPA** for all students whose **CGPA is greater than 8** by using an **INNER JOIN** and the **WHERE** clause in SQLite.